

Advanced DBMS – System Design Discussion Guidelines

Overview

The System Design Discussion component is a core part of the Advanced DBMS course.

The objective of this activity is to help students develop a deeper understanding of how modern database systems are designed internally, why specific architectural decisions were made, and what trade-offs database engineers accept while building production-grade systems.

Unlike traditional programming assignments, the focus here is not on writing code. Instead, students are expected to study real database systems, analyze their architecture, understand their implementation details, and document their findings in a structured and insightful manner.

The final submission will be made as a GitHub Pull Request (PR) containing a README.md document.

Objective

Important

The objective is NOT to:

- Copy documentation from official websites
- Summarize blog posts without analysis
- Generate lengthy documents filled with implementation details without understanding

You are expected to:

- Study the architecture of real database systems
- Understand the motivation behind design choices
- Analyze storage engines and internal components
- Explain trade-offs between alternative approaches
- Connect implementation details to observed system behavior
- Document findings clearly using diagrams, examples, and reasoning

Marks will primarily be awarded based on:

- Conceptual understanding
 - Architectural reasoning
 - Quality of analysis
 - Clarity of explanation
 - Ability to explain trade-offs
-

Topic 1: PostgreSQL vs SQLite Architecture Comparison

Study Focus

Compare PostgreSQL and SQLite on:

- Overall architecture
- Process model
- Client-server vs embedded database design
- Storage engine architecture
- Database file organization
- Table storage
- Page layout
- Disk layout
- Index implementation
- Transaction management
- Concurrency control
- Durability mechanisms

Expected Analysis

Students should explain:

- Why PostgreSQL uses a client-server architecture
- Why SQLite is designed as an embedded database
- Internal storage differences
- Concurrency trade-offs
- Scalability implications
- Real-world use cases for each database

Suggested Questions

- Why does SQLite work well for mobile applications?
 - Why is PostgreSQL preferred for large multi-user systems?
 - What architectural decisions lead to these differences?
-

Topic 2: PostgreSQL Internal Architecture

Study Focus

Explore PostgreSQL internals with emphasis on:

Buffer Manager

Location:

`src/backend/storage/buffer/`

Topics:

- Shared buffers
- Page caching
- Buffer replacement
- Page reads and writes

B-Tree Implementation

Location:

`nbtree`

Topics:

- Index structure
- Index page layout
- Search path
- Insert operations
- Page splits

MVCC

Topics:

- Heap tuple versioning
- xmin / xmax
- Visibility rules
- Snapshot isolation

WAL (Write Ahead Logging)

Topics:

- WAL records
- Durability guarantees
- Crash recovery
- Checkpointing

Recommended Exercise

Run:

EXPLAIN ANALYZE

on a multi-table join query.

Analyze:

- Chosen execution plan
- Planner estimates
- Actual execution statistics
- Statistics used by planner
- Relationship with pg_statistic

Expected Analysis

Students should explain:

- How pages move through the buffer manager
 - How PostgreSQL implements MVCC
 - Why VACUUM is necessary
 - How WAL guarantees durability
 - How query planning relies on collected statistics
-

Topic 3: MySQL / InnoDB Storage Engine

Study Focus

Explore InnoDB architecture with emphasis on:

- Clustered indexes
- Primary key storage
- Secondary indexes
- Buffer pool
- Undo logs
- Redo logs
- Row-level locking
- Gap locks
- Transaction processing

Key Comparison with PostgreSQL

PostgreSQL:

- Append-only heap updates
- Tuple versioning
- VACUUM cleanup
- MVCC through multiple tuple versions

InnoDB:

- In-place updates
- Undo logs
- Oracle-style MVCC
- Clustered storage

Expected Analysis

Students should explain:

- Why clustered indexes improve lookup performance
- Purpose of undo logging
- Purpose of redo logging
- Locking mechanisms
- Isolation levels
- Trade-offs compared to PostgreSQL

Suggested Questions

- Why does InnoDB need both undo and redo logs?
 - What advantages do clustered indexes provide?
 - Why did PostgreSQL choose a different MVCC model?
-

Topic 4: RocksDB Architecture

Study Focus

Explore LSM-tree based storage architecture.

Topics:

- MemTable
- Immutable MemTable
- SSTables
- WAL
- L0 to Ln storage levels
- Bloom Filters
- Compaction
- Read path
- Write path

Recommended Exercise

Run RocksDB benchmark tools and observe:

- Write amplification
- Read amplification
- Space amplification

under different compaction strategies.

Expected Analysis

Students should explain:

- Why LSM trees are optimized for writes

- How data flows through the system
- Why compaction is required
- Read performance trade-offs
- Storage efficiency trade-offs

Suggested Questions

- Why are LSM trees preferred in write-heavy workloads?
 - Why can compaction become expensive?
 - How do Bloom Filters improve read performance?
-

Repository

All submissions must be made to the following repository:

<https://github.com/KnightKnight27/scaler-Adv-DBMS>

Submission Deadline

Deadline

23 June 2026 (11:59 PM IST)

Important Notes:

- All submissions must be completed before the deadline.
 - Late submissions may not be considered for grading.
 - Students are advised to create their Pull Requests well before the deadline to avoid last-minute issues.
-

Submission Process

Students must:

1. Fork the repository.
2. Create the required directory structure.
3. Add their README.md file.
4. Commit changes.
5. Create a Pull Request against the main repository.

The GitHub Pull Request itself will be treated as the final submission.

No separate submission form is required unless communicated later.

Repository Structure

Create the following directory:

System_Design_Docs/

Inside it, create a topic-specific directory.

Example:

System_Design_Docs/

```
|— PostgreSQL_vs_SQLite/  
|  └─ README.md
```

```
|— PostgreSQL_Internals/  
|  └─ README.md
```

```
|— MySQL_InnoDB/  
|  └─ README.md
```

```
└─ RocksDB/  
   └─ README.md
```

Students should place their README.md inside the directory corresponding to their chosen topic.

README.md Requirements

The README should contain the following sections.

1. Problem Background

Explain:

- Why the database system exists
 - What problem it was designed to solve
 - Historical context if relevant
-

2. Architecture Overview

Include:

- High-level architecture diagram
 - Main system components
 - Data flow
-

3. Internal Design

Discuss:

- Storage structures
 - Memory management
 - Index organization
 - Transaction processing
 - Concurrency control
 - Recovery mechanisms
-

4. Design Trade-Offs

Discuss:

- Advantages
- Limitations
- Performance implications
- Engineering decisions

Students are encouraged to compare alternative approaches where relevant.

5. Experiments / Observations

Include practical observations such as:

- Query plans
- Benchmarks
- Performance measurements
- System behavior under different workloads

where applicable.

6. Key Learnings

Summarize:

- Important insights
 - Architectural lessons
 - Surprising observations
 - Practical takeaways
-

Pull Request Guidelines

Branch Naming (Recommended)

Example:

feature/SCALER_12345

Pull Request Title Format

The Pull Request title MUST follow:

SCALER_<ROLL_NUMBER>

Example:

SCALER_24567

This format will be used for grading and tracking submissions.

Evaluation Criteria

Component	Weightage	Evaluation Criteria
Conceptual Understanding	35%	Depth of architectural understanding
Internal Design Analysis	25%	Correct explanation of DB internals
Trade-off Discussion	15%	Ability to reason about design decisions
Experiments & Observations	15%	Quality of practical analysis
Documentation Quality	10%	Clarity, structure, readability, diagrams

Total: 100 Marks

Submission Checklist

Before creating your Pull Request, ensure:

- README.md is present
 - README.md is inside the correct topic directory
 - Repository structure is followed exactly
 - Diagrams are properly referenced
 - Experiments or observations are included where applicable
 - PR title follows the required roll number format
 - Pull Request is raised before 23 June 2026, 11:59 PM IST
-

Important Rules

- Submission link is the GitHub Pull Request itself.
 - Follow the repository structure exactly.
 - README.md should be your original work.
 - Plagiarism may result in penalties.
 - Focus on reasoning and understanding rather than document length.
 - Students may use diagrams, references, and external resources, but all sources should be appropriately credited.
-

Final Reminder

Database systems are collections of engineering trade-offs.

The goal of this exercise is not to memorize implementation details, but to understand:

- Why the system was designed this way
- What alternatives were available
- What trade-offs were accepted
- How those choices impact performance, scalability, durability, and correctness

Focus on architectural reasoning, implementation insights, and clear communication.

A concise, insightful, and well-structured README is significantly more valuable than a lengthy document containing copied material.